

DevSecOps: Integração de Segurança em Pipelines

Automação de segurança do código à infraestrutura para detecção precoce de vulnerabilidades.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Sumário

- A ascensão do DevSecOps e Shift-Left
- Segurança no fluxo CI/CD: camadas estratégicas
- Ferramentas de análise: SAST, SCA, Secret Scanning
- DAST, Container e IaC Scanning
- Desafios, boas práticas e fluxo completo

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



A Ascensão do DevSecOps e o 'Shift-Left'

A integração da segurança nos pipelines de CI/CD marca uma mudança de paradigma, conhecida como '**Shift-Left**'. Em vez de tratar a segurança como uma fase isolada e tardia, o DevSecOps a incorpora proativamente desde as primeiras etapas do ciclo de vida de desenvolvimento de software (SDLC). O objetivo principal é a **detecção precoce de vulnerabilidades**, o que não só reduz significativamente os custos de remediação, mas também minimiza os riscos de explorações em ambientes de produção. A automação é fundamental, permitindo que verificações de segurança sejam executadas a cada *commit*, garantindo uma inspeção contínua do software e promovendo uma cultura de responsabilidade compartilhada.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Segurança no Fluxo CI/CD

1

Fase de Build/Commit

Nesta fase inicial, realizamos SAST (análise estática pré-execução) e Secret Scanning (busca por credenciais expostas), verificações rápidas essenciais para detectar falhas precocemente e alinhadas ao shift-left.

2

Resolução de Dependências

A seguir, focamos na análise de componentes de terceiros por meio de **SCA** (Software Composition Analysis). Esta etapa é crucial para identificar bibliotecas, frameworks e pacotes vulneráveis, bem como riscos de licenciamento, garantindo que dependências introduzidas não comprometam a segurança da aplicação.

3

Pós-Build/Staging

Em staging, realizamos DAST (varredura dinâmica em execução) e testes de penetração automatizados, que simulam ataques externos para identificar vulnerabilidades de runtime e validar a robustez da aplicação antes do deploy final.

4

Infraestrutura/Deployment

Por fim, antes do release, fazemos a varredura de contêineres e IaC para verificar configurações de segurança e prevenir vulnerabilidades de runtime, garantindo uma infraestrutura de produção segura e resiliente.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



SAST: Análise Estática

O **SAST** (Static Application Security Testing) analisa o código-fonte ou binários sem executar a aplicação, buscando falhas lógicas, injeção de SQL e XSS. Ele oferece feedback imediato aos desenvolvedores, apontando a linha exata do problema. Seu foco é garantir padrões de segurança antes da compilação, embora possa gerar falsos positivos.



INSTITUIÇÃO EXECUTORA



COORDENADORA

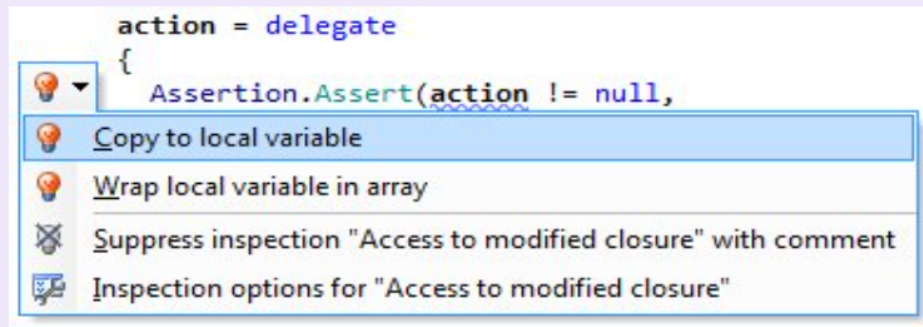


APOIO



Ferramentas Líderes em SAST

A escolha da ferramenta SAST (Static Application Security Testing) ideal depende diretamente da *stack* tecnológica e das necessidades de integração do projeto. Entre as soluções de destaque, **SonarQube** é amplamente utilizado para garantir a qualidade e segurança do código, com forte integração a plataformas como Jenkins e Azure DevOps, focando em 'Clean Code'. O **Snyk** ganhou tração por ser 'developer-friendly', integrando-se diretamente ao fluxo de trabalho do desenvolvedor e oferecendo sugestões automáticas de correção. Já o **Semgrep** é um motor de análise estática ultrarrápido que utiliza regras baseadas em padrões de código, sendo ideal para customização e pipelines de alta velocidade. A adoção dessas ferramentas automatiza a revisão de código, liberando os auditores de segurança para focarem em problemas arquiteturais mais complexos.



ReSharper: sugestões de código para integração SAST.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



SCA: Análise de Composição de Software

- **Prevenção de Vulnerabilidades:** O SCA é essencial para identificar e mitigar riscos de segurança introduzidos por até 80-90% do código proveniente de bibliotecas de terceiros (Open Source) em aplicações modernas, evitando a herança de vulnerabilidades de pacotes baixados via npm, maven, pip ou nuget.
- **Rastreamento de Dependências:** Tem como principal objetivo mapear todas as dependências do projeto, incluindo as diretas (as que o projeto usa diretamente) e as *transitivas* (dependências das dependências), garantindo uma visão completa da composição do software.
- **Identificação de CVEs:** Essa ferramenta cruza a lista detalhada de bibliotecas utilizadas com bases de dados de vulnerabilidades conhecidas, como a NVD (National Vulnerability Database), para alertar sobre quaisquer Common Vulnerabilities and Exposures (CVEs) associadas.
- **Compliance de Licenças:** Além da segurança, o SCA verifica a conformidade das licenças das bibliotecas (por exemplo, GPL vs. Apache) para assegurar que elas são compatíveis com o modelo de negócio da empresa e evitar problemas legais.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Ferramentas para SCA

As ferramentas de *Software Composition Analysis* (SCA) automatizam a identificação de dependências em projetos, verificando vulnerabilidades conhecidas. Sua integração nos pipelines de CI/CD é crucial para manter a cadeia de suprimentos de software segura, prevenindo falhas como o Log4Shell.



Dashboard de segurança: monitorando dependências e vulnerabilidades em tempo real.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Secret Scanning: Prevenção

O Secret Scanning foca em impedir o vazamento de credenciais (secrets), como chaves de API e senhas, no controle de versão. Uma vez em um repositório, uma chave deve ser considerada comprometida. A implementação ocorre via pre-commit hooks ou scanning no servidor de CI. Ele utiliza expressões regulares e entropia para identificação.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Comparativo: Proteção de Segredos

Esta tabela compara as ferramentas Gitleaks e GitGuardian, detalhando seus focos principais e usos típicos na proteção contra vazamento de segredos em ambientes de desenvolvimento e produção.

Ferramenta	Foco Principal	Uso Típico
Gitleaks	Engine open-source ultrarrápida.	Integrado em pipelines de CI como etapa de 'build fail' e varredura de histórico.
GitGuardian	Plataforma corporativa de monitoramento.	Monitoramento em tempo real de repositórios públicos e privados com alertas para times de SOC.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



DAST: Análise Dinâmica de Segurança

Dando continuidade à nossa discussão sobre segurança de aplicações, enquanto o SAST se concentra no código estático, o **DAST (Dynamic Application Security Testing)** atua na aplicação 'viva', em execução. Ele adota uma perspectiva de '*Black Box*', simulando ataques que um *hacker* externo faria para identificar vulnerabilidades. O DAST é crucial para validar a eficácia das proteções implementadas em tempo de execução.

- **Onde atua:** Testa componentes críticos como *endpoints* de API, formulários de entrada, *headers* de resposta e configurações de servidor, abrangendo a superfície de ataque que um usuário ou um agente malicioso interagiria.
- **O que detecta:** É eficaz na identificação de erros de configuração de segurança, vulnerabilidades de sessão, falhas em mecanismos de autenticação e autorização, e problemas de infraestrutura que só se manifestam com a aplicação em funcionamento.
- **Requisito:** Diferentemente do SAST, que opera no código-fonte, o DAST necessita de um ambiente funcional (como um ambiente de *Staging* ou QA) para rodar os testes de forma eficaz, permitindo a interação real com a aplicação.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



OWASP ZAP e Stackhawk

A automação de *DAST* (Dynamic Application Security Testing) em pipelines é um desafio pela duração da execução, mas fundamental para a segurança. Ferramentas como o **OWASP ZAP** (Zed Attack Proxy), a mais popular para varreduras básicas ou ataques ativos contra novos deploys via Docker, e o **Stackhawk**, que moderniza o DAST para desenvolvedores e pode 'quebrar o build' ao encontrar novas vulnerabilidades em runtime, são essenciais. A estratégia comum é executar um *scan* rápido (baseline) no pipeline, reservando varreduras completas para horários de menor movimento para não atrasar o deploy.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Container e IaC Scanning

- A análise de **Container Scanning** examina imagens Docker, buscando vulnerabilidades no sistema operacional base (como bibliotecas desatualizadas do Debian/Alpine) e configurações potencialmente inseguras (por exemplo, a execução como usuário *root*).
- Já o **IaC Scanning** (Infraestrutura como Código) verifica arquivos de configuração como Terraform, CloudFormation ou Kubernetes YAML, identificando riscos antes da implantação da infraestrutura.
- Entre os **riscos comuns** detectados estão *buckets* S3 com acesso público, grupos de segurança que expõem a porta 22 para a internet, ou políticas de IAM com permissões excessivas.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Trivy e Checkov: Segurança para Contêineres e IaC

Trivy: Scanner Abrangente

O Trivy é um **scanner de segurança** abrangente e de código aberto, amplamente utilizado para detectar vulnerabilidades em contêineres e outros artefatos. Ele é **extremamente simples de usar** em pipelines de CI/CD, com um comando direto como ``trivy image [imagem]``. Ele identifica falhas em pacotes de sistemas operacionais (OS packages) e dependências de linguagens de programação, oferecendo uma visão detalhada dos riscos. É possível configurar **limites de severidade** (por exemplo, falhar o build apenas para vulnerabilidades 'Critical' ou 'High') para otimizar o fluxo de trabalho.



Checkov: Análise de IaC

O Checkov foca na **Análise Estática de Código** (Static Code Analysis - SCA) para *Infraestrutura como Código* (IaC). Ele possui **centenas de políticas pré-definidas** para os principais provedores de nuvem, como AWS, Azure e GCP, garantindo que o provisionamento de infraestrutura siga as melhores práticas de segurança e conformidade. Ao integrar o Checkov no pipeline, ele pode detectar configurações inadequadas ou violações de segurança antes que a infraestrutura seja provisionada, permitindo que as equipes definam **limites para falhas no build** com base na severidade das políticas violadas.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



O Fluxo DevSecOps Completo

1

Na IDE (Pré-commit)

Nesta fase inicial, são integrados plugins de SAST (Static Application Security Testing) como Snyk ou SonarLint diretamente no ambiente de desenvolvimento.

Adicionalmente, *hooks* de *Secret Scanning* são configurados para fornecer feedback imediato ao desenvolvedor, prevenindo a inclusão de segredos no código-fonte antes mesmo do commit.

2

No Pull Request

Durante a revisão de código, uma varredura SAST incremental é executada, focando apenas nas alterações introduzidas no *pull request*. Simultaneamente, ocorre a análise SCA (Software Composition Analysis) para identificar vulnerabilidades em novas dependências adicionadas, garantindo que apenas código seguro seja mesclado à base principal.

3

Na Integração Contínua

Após a aprovação do *pull request*, o processo de integração contínua inicia, onde a imagem do artefato é construída. Ferramentas como Trivy são utilizadas para *Container Scan* e Checkov para *IaC Scan* (Infrastructure as Code Scan), garantindo que tanto as imagens quanto a infraestrutura definida em código estejam livres de vulnerabilidades de forma automatizada.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



O Fluxo DevSecOps Completo

4

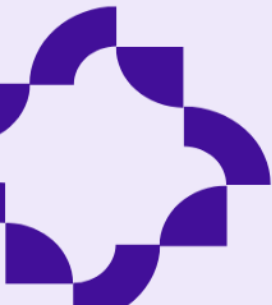
Na Implantação Contínua

Após as verificações de integração contínua, o deploy da aplicação é realizado em um ambiente de *Staging*. Em seguida, uma varredura *DAST* (Dynamic Application Security Testing) é executada, tipicamente com ferramentas como OWASP ZAP, para validar a segurança da aplicação em tempo de execução, simulando ataques e identificando falhas que só se manifestam em um ambiente ativo.

5

No Monitoramento Contínuo

Mesmo após o deploy em produção, o ciclo de segurança não termina. O *Monitoramento Contínuo* envolve a varredura constante de vulnerabilidades que podem surgir após a implantação, como novas *CVEs* (Common Vulnerabilities and Exposures) ou configurações incorretas, garantindo a proteção contínua da aplicação e da infraestrutura.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Desafios e Boas Práticas na Integração de Segurança

- Evite o '**Build Fatigue**': A sobrecarga de ferramentas pode impactar significativamente a velocidade do pipeline, exigindo a adoção de varreduras incrementais ou paralelas para otimizar o tempo.
- Atenção aos **Falsos Positivos**: Ferramentas que geram muitos alertas irrelevantes rapidamente perdem a credibilidade; é crucial ajustar as regras de detecção para o contexto específico do negócio, priorizando ameaças reais.
- Cultive uma **Cultura de Segurança**: A equipe de segurança deve atuar como um *facilitador*, empoderando os desenvolvedores com as ferramentas e o conhecimento necessários para que se tornem autônomos na identificação e correção de vulnerabilidades.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Qual ferramenta é MAIS adequada para detectar uma configuração insegura em um arquivo Terraform que deixaria um banco de dados exposto à internet?

- A Checkov
- B Gitleaks
- C SonarQube
- D OWASP ZAP

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Qual ferramenta é MAIS adequada para detectar uma configuração insegura em um arquivo Terraform que deixaria um banco de dados exposto à internet?

- ✓ Checkov
- ✗ Gitleaks
- ✗ SonarQube
- ✗ OWASP ZAP



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Verdadeiro ou falso?

O DAST (Dynamic Application Security Testing) é executado diretamente no código-fonte antes da aplicação ser compilada.

← Falso

Verdadeiro →

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



× Falso

O DAST (Dynamic Application Security Testing) é executado diretamente no código-fonte antes da aplicação ser compilada.

Explicação

O DAST atua sobre a aplicação em execução, simulando ataques externos para identificar vulnerabilidades, enquanto a análise de código-fonte pré-compilação é realizada pelo SAST.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Verdadeiro ou falso?

O Secret Scanning serve para detectar vulnerabilidades de Injeção de SQL no código-fonte da aplicação.

← Falso

Verdadeiro →

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



× Falso

O Secret Scanning serve para detectar vulnerabilidades de Injeção de SQL no código-fonte da aplicação.

Explicação

O Secret Scanning foca na identificação de credenciais expostas, como chaves de API, senhas e tokens em repositórios, enquanto as vulnerabilidades de Injeção de SQL são tipicamente detectadas por ferramentas SAST ou DAST.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Verdadeiro ou falso?

O SCA (Software Composition Analysis) é fundamental para detectar vulnerabilidades como o Log4Shell em bibliotecas de terceiros e componentes open-source.

← Falso

Verdadeiro →

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



✓ Verdadeiro

O SCA (Software Composition Analysis) é fundamental para detectar vulnerabilidades como o Log4Shell em bibliotecas de terceiros e componentes open-source.

Explicação

O SCA é projetado especificamente para identificar e gerenciar riscos de segurança associados a dependências externas, bibliotecas de terceiros e componentes de código aberto, como o caso do Log4Shell.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Conclusão

- DevSecOps "Shift-Left" integra segurança desde o início do SDLC, reduzindo drasticamente custos de remediação e riscos.
- Ferramentas como SAST (SonarQube) e SCA (Snyk) atuam no código-fonte e dependências, antecipando vulnerabilidades.
- DAST (OWASP ZAP) testa a aplicação em execução, complementando análises estáticas com validações de runtime.
- Scanning de Container (Trivy) e IaC (Checkov) é crucial para proteger a infraestrutura que suporta o software.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

